Laboratório de Desenvolvimento para Dispositivos Móveis

PUC Minas

Prof. Cristiano Rodrigues

Introdução ao Flutter

O que é Flutter?

SDK para criação de Apps 2D

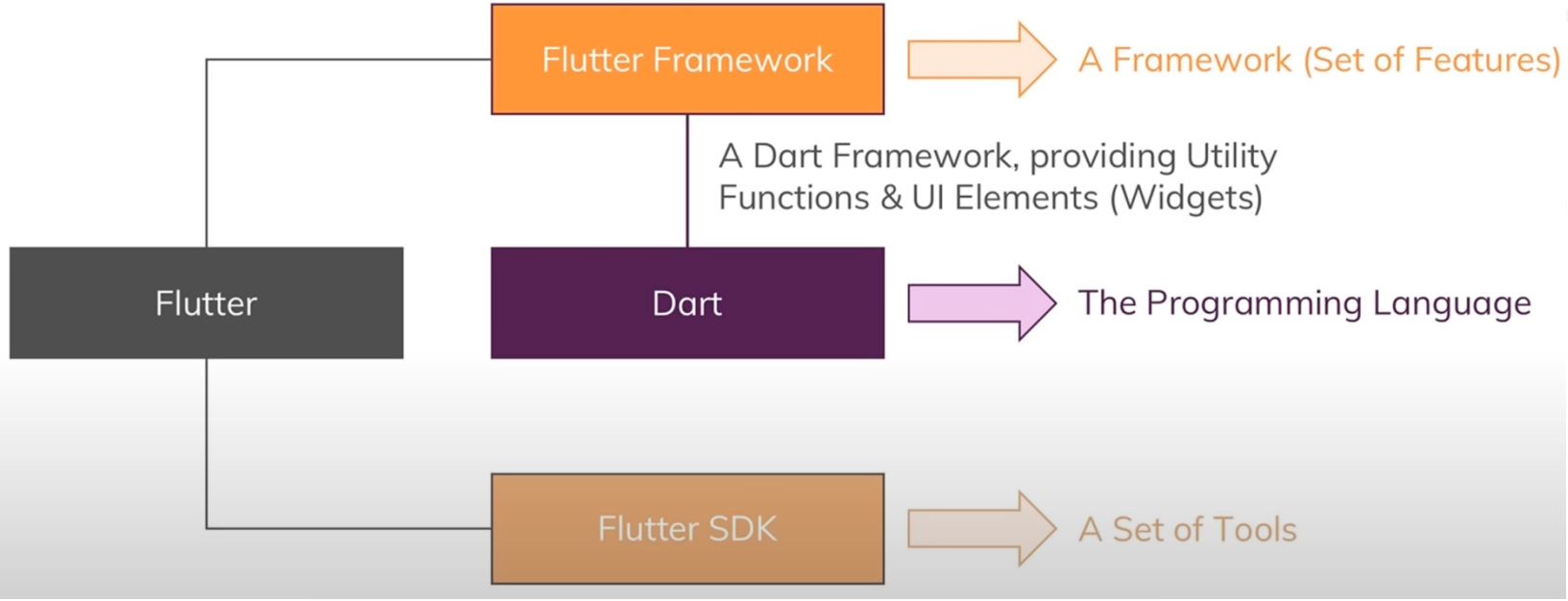
Framework de programação construído em Dart

Escreva código com UMA linguagem (Dart)

Personalize para diferentes plataformas (iOS / Android)

Construa e publique

O que é Flutter?



O que é Flutter?

- É um framework construído pelo Google para o desenvolvimento de multiplataforma
- Como o mesmo **código base**, gera aplicativos **nativos** para Android, IOS, Web e Desktop
- Utiliza o Dart como linguagem de desenvolvimento

Por que Flutter?

- Criado e mantido pelo Google
- Mais de 170 widgets (componentes) prontos para serem utilizados
- Principais plugins para acesso aos recursos nativos no celular também são mantidos pelo Google

Por que Flutter?

- Flutter desenha todos os pixels na tela, tornando o aplicativo altamente customizável
- Alta performance: aplicativos rodam em 60 frames por segundo (ou até 120 caso o aparelho suporte)

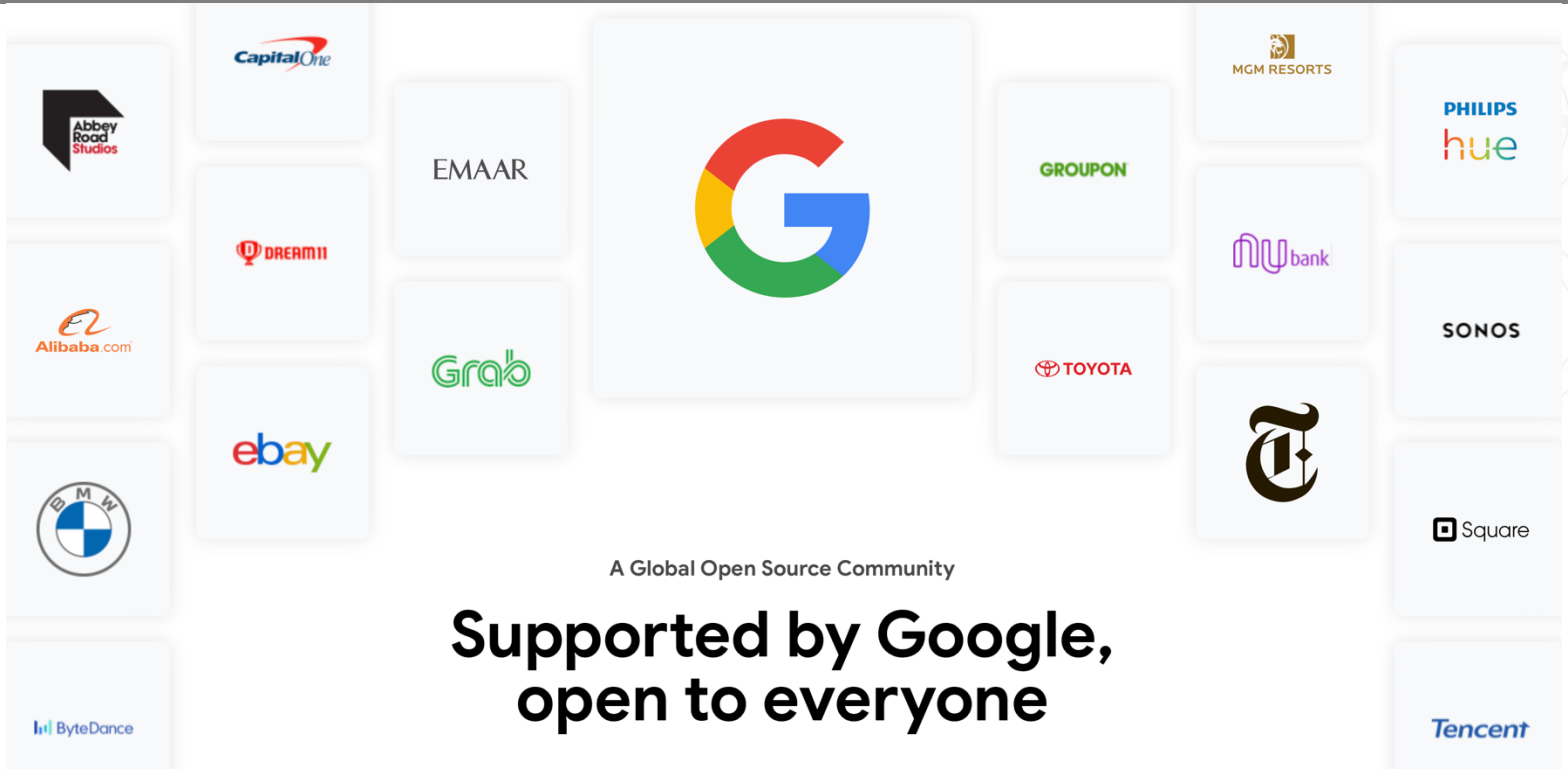
Por que Flutter?

- Alta produtividade: alterações no código refletidos no celular ou emulador em até 0,5 s
- Baseado em **três** pilares:
    1. Desenvolvimento rápido
    2. Interfaces bonitas
    3. Performance nativa

Por que Flutter?

- Alta produtividade: alterações no código refletidos no celular ou emulador em até 0,5 s
- Baseado em **três** pilares:
    1. Desenvolvimento rápido
    2. Interfaces bonitas
    3. Performance nativa

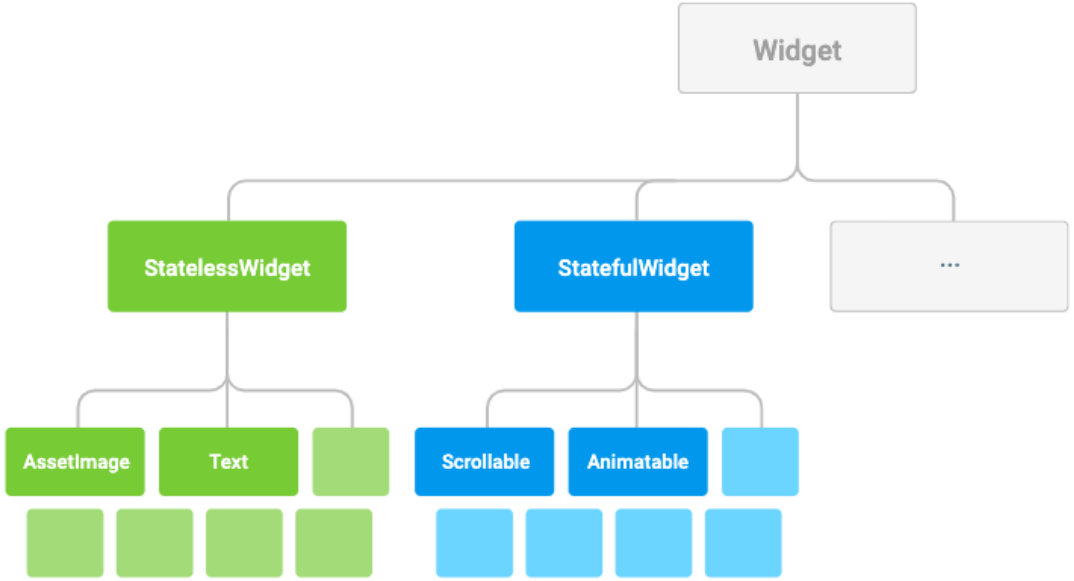
Quem está usando o Flutter?



Fonte: https://flutter.dev/

Flutter - Widgets

No Flutter, tudo é considerado um Widget, desde um tema para sua aplicação, como também um componente mais complexo



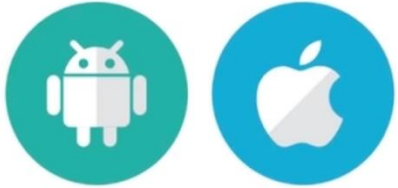
Fonte: https://flutter.dev/

Flutter - Widgets

- Diferente de outras estruturas que separam visualizações, controladores de exibição, layouts e outras propriedades, o Flutter possui um modelo de objeto unificado e consistente: o **widget**

Flutter

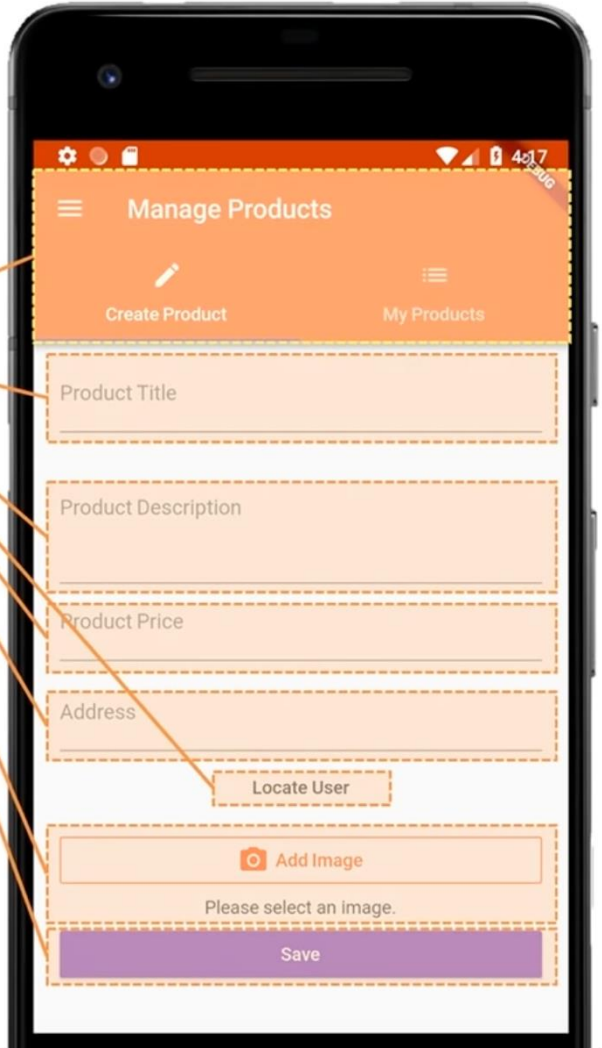
Construa uma árvore de widgets



Diferenças entre plataformas

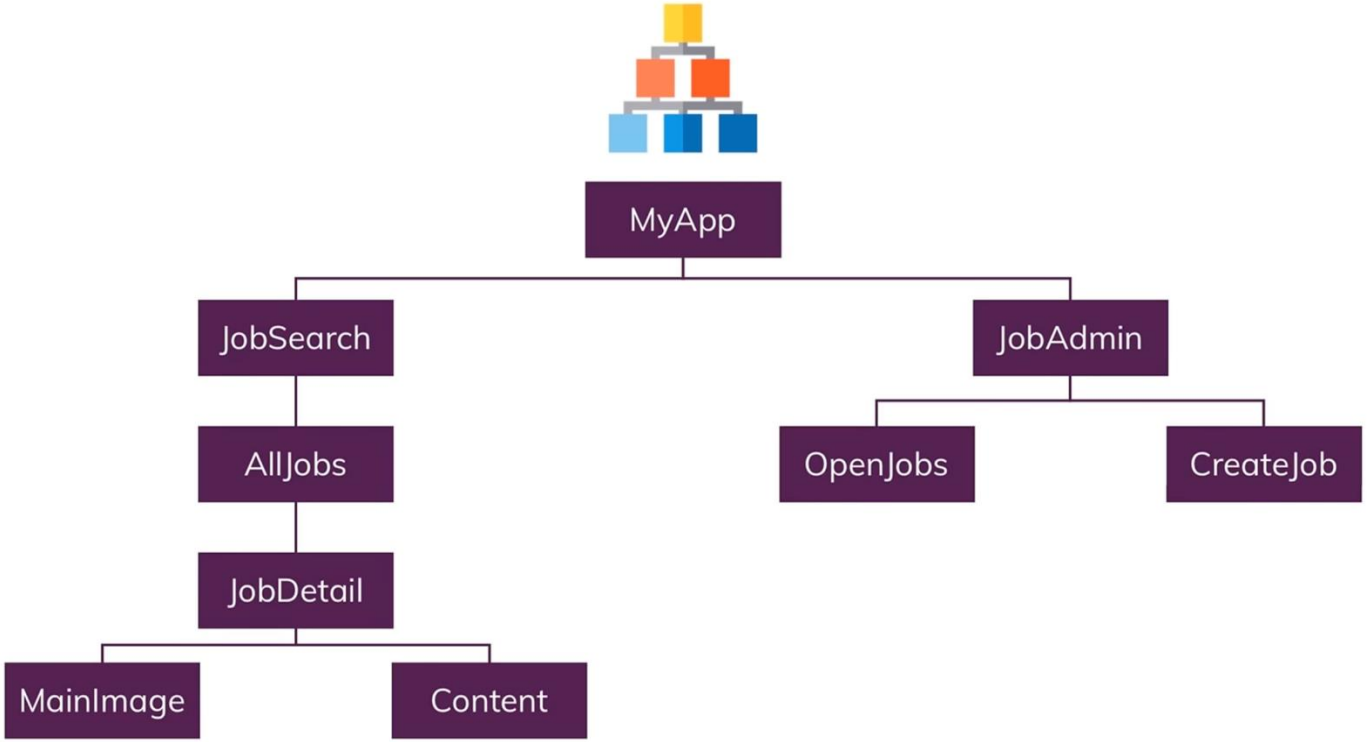
Base de código única

Widget!

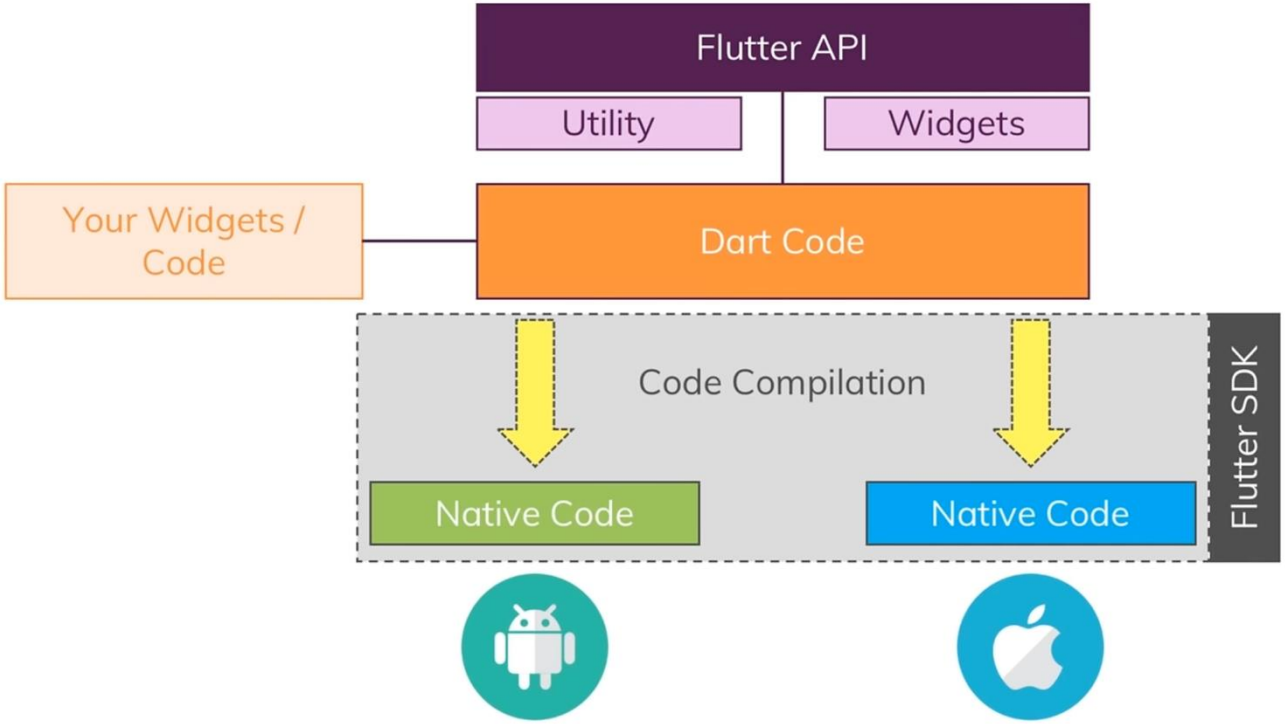


PUC Minas

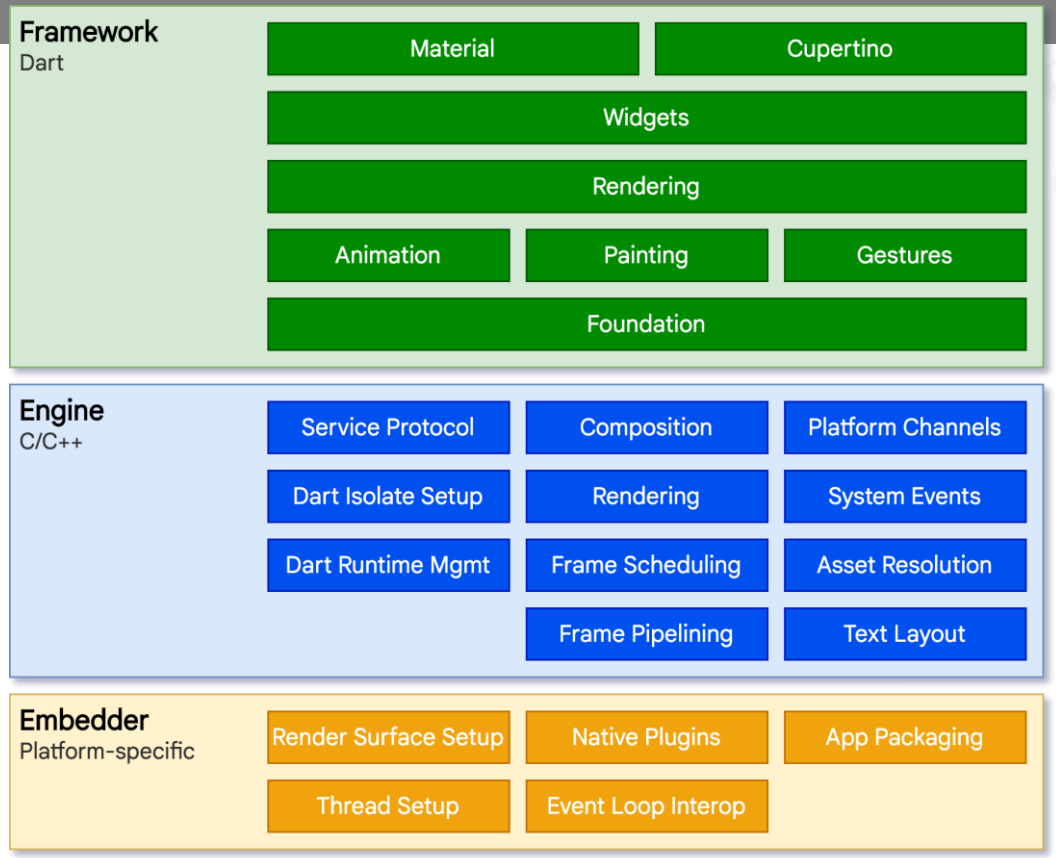
Construa um árvore de widgets



Como Flutter / Dart gera um App nativo



Flutter - Arquitetura



Fonte: https://flutter.dev/docs/resources/technical-overview

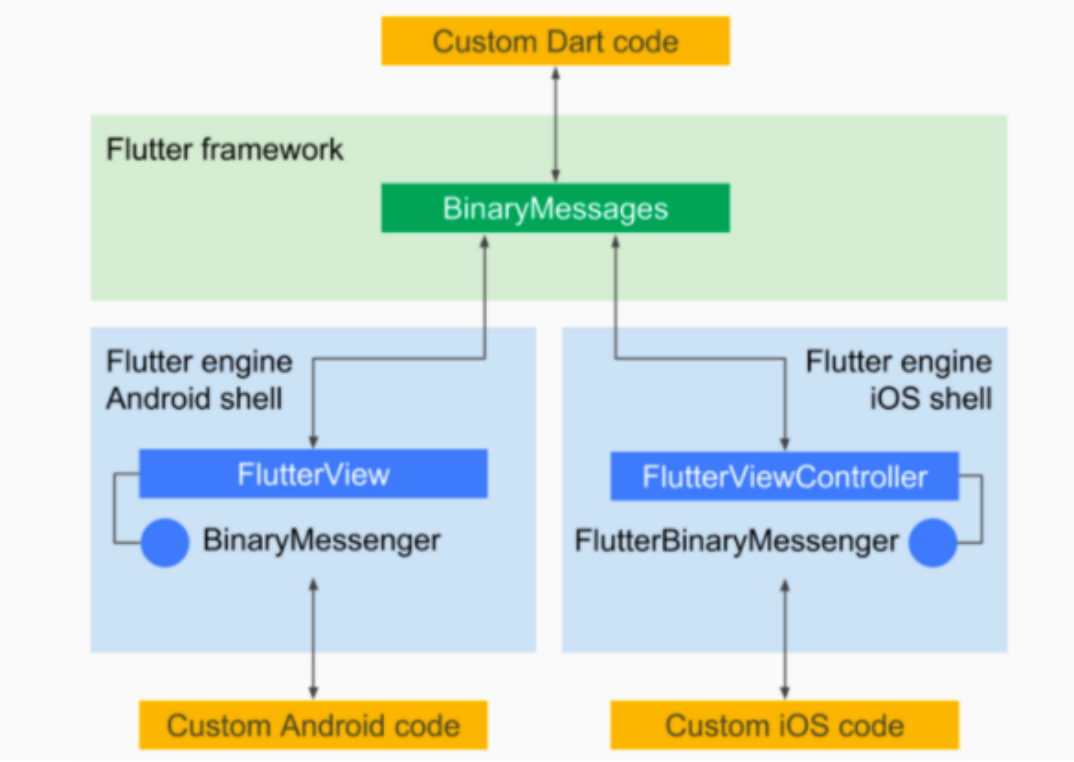
Flutter - Arquitetura

- A primeira camada é responsável por todo o Framework Flutter escrito em Dart, onde vamos trabalhar
- A segunda camada se trata do core do Flutter, lá são feitos tratamentos específicos de cada sistema e a engine gráfica

Flutter - Arquitetura

- A terceira camada é responsável por incorporar o Flutter em várias plataformas, suas principais tarefas são: renderizar configurações de superfície, configurações de thread e plugins. Ou seja, facilitar a comunicação com a plataforma Android e IOS.

Flutter - Arquitetura



Fonte: https://developers-br.googleblog.com/2018/09/canais-de-plataforma-do-flutter.html

Referências Bibliográficas

- **Flutter:** https://flutter.dev/
- **Guia de Flutter:** https://www.devmedia.com.br/guia/flutter/40713
- **Dart:** https://dart.dev/
- **Curso de Dart Gratuito** no Udemy: https://www.youtube.com/watch?v=vI4wfJSvomY

PUC Minas

Criando um projeto Flutter

Criando um projeto no Flutter

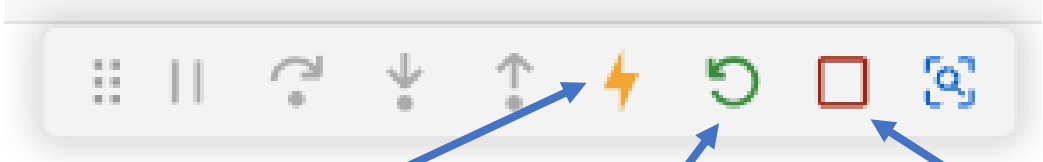
- Abrir VS Code
- Instalar extensão Flutter. A extensão Dart é instalada junto coma do Flutter
- View > Command Palette > Flutter: New Project > Application
- Crie/escolha um diretório para salvar o projeto
- Escolha um nome para o projeto e pronto! Será criado um projeto Flutter
- O arquivo main.dart contido na pasta lib contém um exemplo de um aplicativo

Criando um projeto no Flutter

- No canto inferior direito, escolha um dispositivo (*device*). Por exemplo, o Chrome
- Em seguida, no canto superior direito, clique no símbolo de play: *Run Without Debugging*
- Será aberta um navegador do Chrome com a simulação do aplicativo escrito no arquivo main.dart
- O App de exemplo mostra na tela quantas vezes o usuário clicou no botão +

Criando um projeto no Flutter

Caso realize pequenas alterações no código e queira ver de forma rápida como a alteração refletiu no App, utilize o botão *Save and Hot Reload*



Save and Hot Reload *Restart* *Stop*

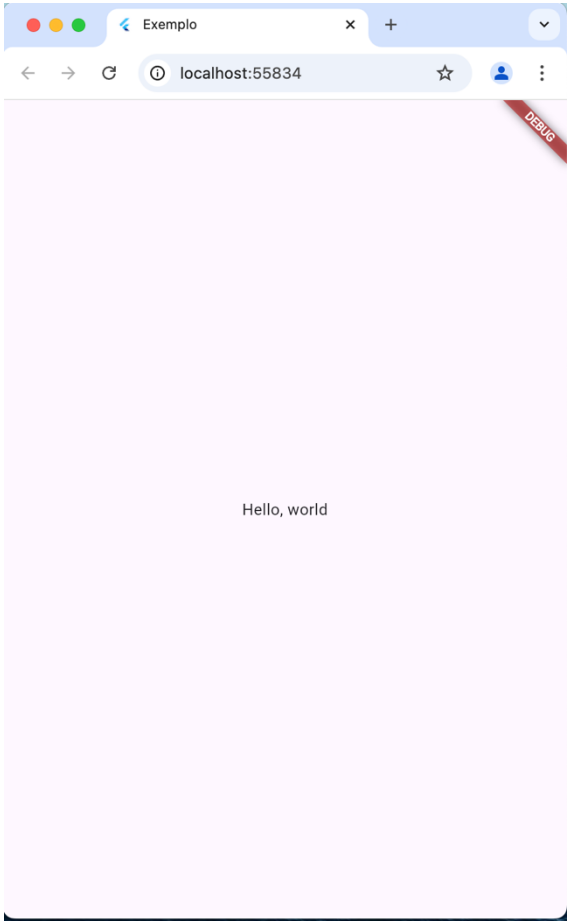
Fazendo ajustes no código

- No arquivo main.dart, localize o MaterialApp e, dentro de MaterialApp altere a cor (`seedColor`) e também o título da home (`MyHomePage(title: …)`)

- Utilize o botão *Save and Hot Reload* para visualizar as alterações no App.

Hello, World!

- Na pasta lib, altere o código no arquivo main.dart para exibir o App ao lado.



Hello, world!

```
import 'package:flutter/material.dart';

void main() {
runApp(const MaterialApp(
  title: 'Exemplo’,
  home: Scaffold(
    body: Center(
      child: Text("Hello, world")))));
}
```

final vs. const

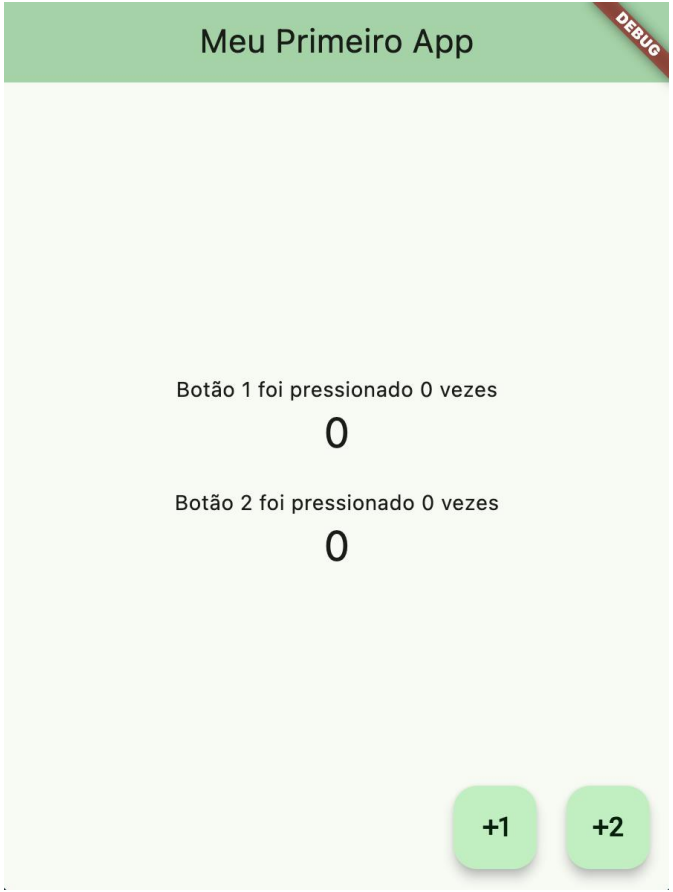
Use **final** quando o valor é atribuído uma vez, mas só é conhecido em tempo de execução.

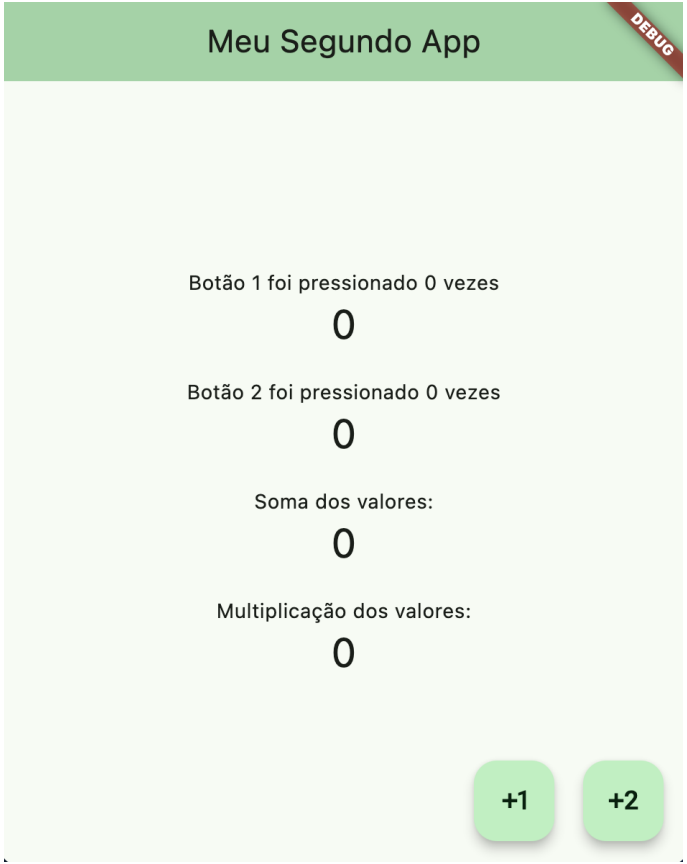
Use **const** quando o valor é imutável e definido em tempo de compilação.

Objetos criados com **const** são mais eficientes em termos de memória, já que eles podem ser reutilizados em diferentes partes do código.

Prática 1

A partir do exemplo de aplicativo gerado pelo Flutter, implemente o aplicativo mostrado.



Prática 2

A partir do exemplo de aplicativo gerado pelo Flutter, implemente o aplicativo mostrado.